

REDROB HACKATHON

Ranking Candidates the Way a Great Recruiter Would

A hybrid scoring system that reads career history as evidence and treats listed skills as claims that need corroboration — built and validated against the full 100,000-candidate pool, entirely offline.

Keyword counting is the wrong answer — and the dataset proves it

What the JD says, in its own words:

“The ‘right answer’ is not ‘find candidates whose skills section contains the most AI keywords.’ That’s a trap we’ve explicitly built into the dataset.”

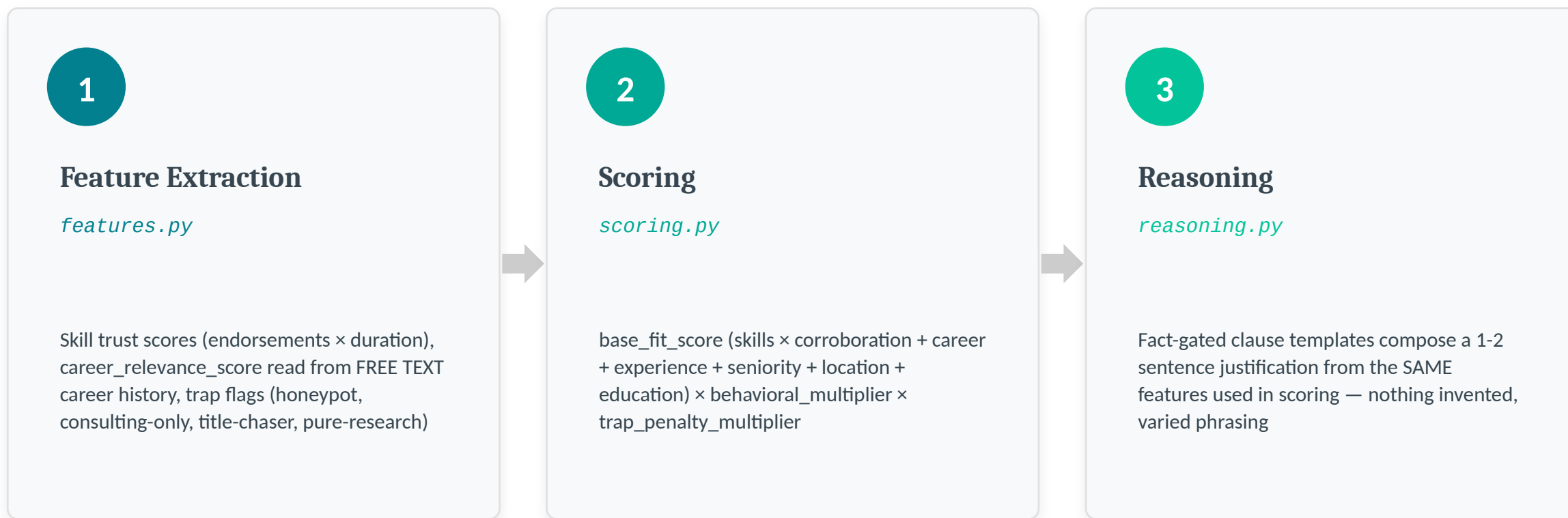
“A candidate who has all the AI keywords listed as skills but whose title is ‘Marketing Manager’ is not a fit, no matter how perfect their skill list looks.”

The bundle's own `sample_submission.csv`:

- Rank 1: HR Manager, 6.1 yrs, “9 AI core skills”
- Rank 4: Content Writer, 8.3 yrs, “8 AI core skills”
- Rank 6: Graphic Designer, 10.4 yrs, “9 AI core skills”

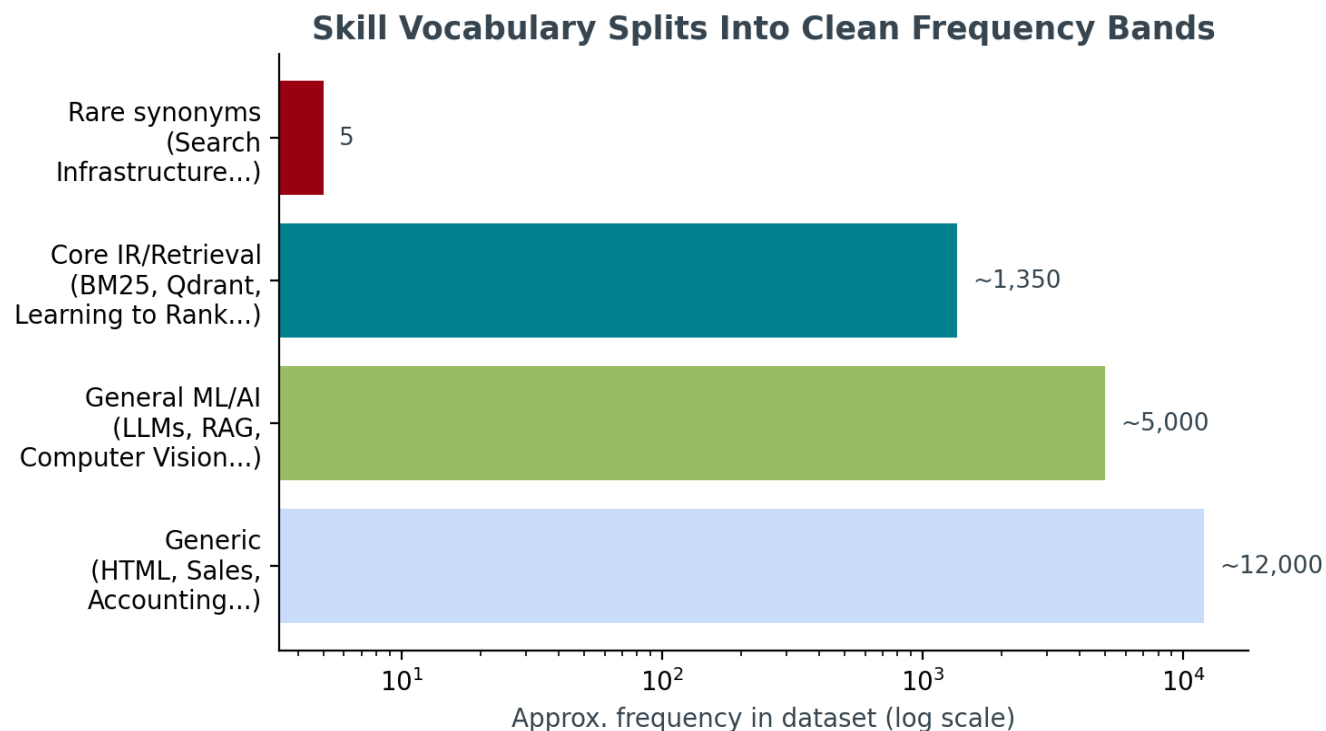
We checked: this is the format reference, and it's a pure keyword-stuffing ranker — exactly the failure mode the JD warns against, sitting in the bundle as a cautionary example.

Three layers: fit, availability, and disqualifiers



`candidates.jsonl(.gz) → extract_features() → score_candidate() → generate_reasoning() → outputs/submission.csv`

The skill vocabulary itself reveals the taxonomy



What this told us:

- Generic skills (HTML, Sales, Accounting...) appear ~12,000× — irrelevant to this JD.
- General ML/AI skills (LLMs, RAG, CV...) appear ~5,000× — relevant but not the JD's specific ask.
- Core IR/retrieval skills (BM25, Qdrant, Learning to Rank...) appear ~1,350× — exactly the JD's “things you absolutely need.”
- A handful of rare synonyms (“Search Infrastructure”, “Ranking Systems”) appear <10× — read as deliberately planted plain-language alternatives.

Skills are claims. Career history is evidence.

What we found during testing: CAND_0000021

14.5-year Project Manager at Wipro. Skills list: Recommendation Systems, Embeddings, Vector Search — each with plausible endorsement counts (3–4) and durations (13–18 months). Every career_history entry: brand design, mechanical engineering, sales, customer support. Zero ML/AI work, ever.



The fix: skill-career corroboration gate

Skill substance is only counted at full value if career_relevance_score (computed independently, purely from career_history TEXT) clears a threshold. Below it, claimed skills are discounted to as little as 25% of their trust-weighted value — endorsement/duration numbers can look reasonable on a hobbyist's skill entry, but the career narrative can't fake substance the same way.

Verified impact: this candidate's rank moved from #8 of 50 to outside the top 100 of 100,000.

Every JD-named trap, operationalized

Honeypots

“Expert”/“advanced” proficiency claimed with ≤ 3 months' use, or ≥ 8 simultaneous “expert” skills.

21/21 matches in EDA were off-domain titles (HR, Accounting, Civil Engineer) — consistent with a planted signature.

Consulting-only

EVERY company in career_history (not just current) is TCS/Infosys/Wipro/Accenture/Cognizant/Capgemini/HCL.

JD explicitly exempts candidates currently at one of these firms with prior product-company experience — our check honors that.

Title-chasers

≥ 3 roles with average tenure < 18 months.

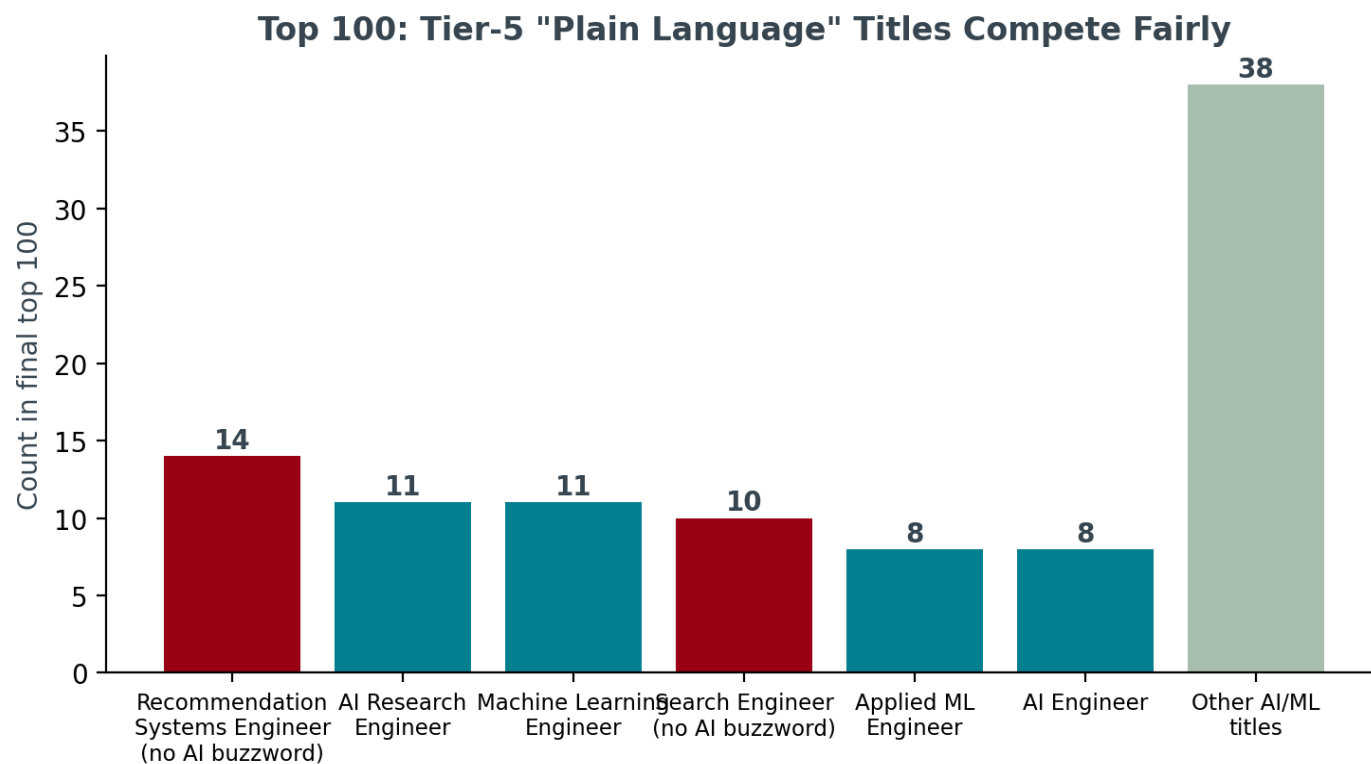
32 AI-titled candidates matched this pattern in the full pool during EDA.

Pure research

Every role title is research-only AND no production-deployment language anywhere in career text.

Both conditions required, so a Research Scientist who later shipped something isn't penalized.

“Tier 5” candidates compete fairly in the final top 100



Why this matters:

career_relevance_score is computed purely from career_history description text — never from the title field. It scans for production/eval/scale language (shipped, NDCG, p95, QPS) and IR domain terms (retrieval, ranking, embeddings, click-through).

This is what lets a “Search Engineer” or “Recommendation Systems Engineer” — titles with zero AI buzzwords — outrank a generic “ML Engineer” whose actual work history is thin.

24 of the final top 100 carry no AI/ML buzzword in their title.

A perfect-on-paper ghost is, for hiring, not available

Six signals feed the multiplier:

- Recency of last_active_date (32% weight)
- Recruiter response rate (28%)
- Open-to-work flag (14%)
- Profile completeness (12%)
- Interview completion rate (8%)
- Email/phone verification (6%)

Why a multiplier, bounded [0.55, 1.10] — not an additive term:

The JD says “down-weight appropriately,” not “disqualify.” A multiplier lets engagement meaningfully move a candidate without letting it single-handedly override a genuinely strong fit — a great-but-ghosted candidate still outranks a mediocre, highly-responsive one if the underlying fit gap is large enough.

*base_fit_score × behavioral_multiplier ×
trap_penalty_multiplier = final score*

Reasoning text: fact-gated, varied, rank-consistent

No LLM calls at ranking time (banned by the compute budget). Instead, every reasoning clause is conditioned on a real boolean/numeric feature from the candidate's own profile, with phrasing variants chosen by a seeded RNG.

Rank 7 • Score 0.9544

“Senior NLP Engineer (7.8 yrs) at Niramai: career history shows shipped production systems, built evaluation/NDCG-style rigor, operated at real scale directly in ranking/retrieval work; 9 core IR/retrieval skills backed by real endorsements and tenure, not just listed; 7.8 yrs experience sits in the JD's target band.”

Rank 100 • Score 0.8634

“2.8-yr NLP Engineer, currently at Meesho: career history shows shipped production systems, built evaluation/NDCG-style rigor directly in ranking/retrieval work; 7 core retrieval-relevant skills with credible endorsement/tenure backing; however only 2.8 yrs experience, below the JD's 5-9 yr band.”

Both real strengths AND honest concerns, stated with specific facts (years, company, skill counts, signal values) — exactly what submission_spec.md Section 3 checks for.

A bug we caught: reasoning that contradicted the score

What we saw while reading ranks 88–100 manually:

Rank 91, score 0.8689 (near the top of an already-curated top 100): “...only marginal signal for this JD; included as lower-confidence filler.” The candidate actually had 5 real strengths and zero concerns — the text was simply wrong.

Root cause and fix:

The original code branched its tone on RANK NUMBER (“if rank > 70, lead with a concern”) — but rank position in a curated top 100 isn't a reliable signal of quality; it just means 90 peers scored higher. Fixed by branching on actual strengths/concerns CONTENT instead. Verified by regenerating and re-reading the same rows.

Full writeup with before/after: DEVELOPMENT_LOG.md in the repo.

FINAL NUMBERS

Results on the full 100,000-candidate pool

0%

Honeypot rate in top 100

limit: $\leq 10\%$

0

Consulting-only careers in top 100

JD-named disqualifier

0

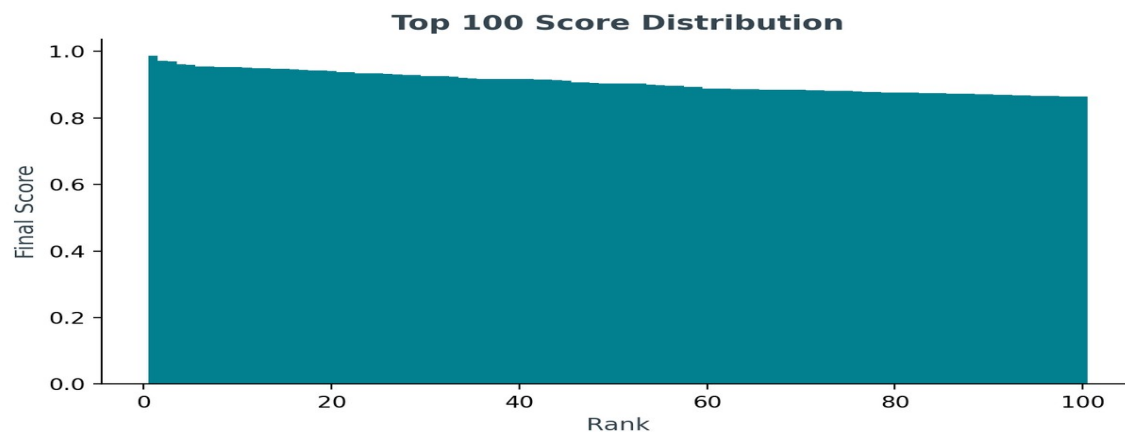
Off-domain titles in top 100

HR, Accounting, Sales, etc.

24

“Tier 5” plain-language titles in top 100

Search/Recsys Engineer



Compute footprint (full 100K pool):

- Runtime: ~51–61 seconds
- Peak RAM: ~1.8 GB
- CPU-only, zero network calls
- Pure Python stdlib (no model weights, no embedding service)

HONEST LIMITATIONS

- `career_relevance_score` is a term-density heuristic over career text, not a learned semantic model — it can miss novel phrasings of the same work.
- `current_role_codes` / `is_pure_research` are inferred from title and description patterns; the dataset has no ground-truth “last wrote code” field.
- Honeypot detection catches the specific “expert-with-near-zero-duration” signature found during EDA — not guaranteed to catch every one of the ~80 honeypots described in the README.

github.com/Harya018/redrob-ranker

`rank.py` → `src/{config,features,scoring,reasoning}.py` → `outputs/submission.csv`
`README.md` (architecture) · `DEVELOPMENT_LOG.md` (bugs found+fixed) · `tests/` (12 unit tests) ·
`sandbox/app.py` (Streamlit demo)

Built and validated end-to-end against the full 100,000-candidate pool with Claude as a development collaborator — declared honestly in `submission_metadata.yaml`.